

Enterprise Certification Framework — Complete User Guide

Version: 2.0 · **Updated:** June 2026

Framework Location: `D:\claude\CompareSkill\`

****Skills Available:**** `/discover-app` · `/application-certification` · `/migration-certification` · `/test-data-generator` · `/production-readiness-review` · `/generate-jira-bugs` · `/generate-pdf-report`

Table of Contents

1. [What This Framework Does](#)
2. [When to Use Each Skill](#)
3. [Prerequisites & Setup](#)
4. [How to Write Prompts — The Golden Rules](#)
5. [Scenario A — Discover a Single Application](#)
6. [Scenario B — Certify One URL, One User](#)
7. [Scenario C — Certify One URL, Multiple Users/Roles](#)
8. [Scenario D — Migration Check \(Old URL vs New URL, Same Role\)](#)
9. [Scenario E — Migration Check with Multiple Roles](#)
10. [Scenario F — Test a Complete Application \(Multiple Base URLs\)](#)
11. [Scenario G — Test Specific Pages Only](#)
12. [Scenario H — Read-Only / Safe Check \(No Data Mutations\)](#)
13. [Scenario I — Resume an Interrupted Run](#)
14. [Scenario J — Quick Check vs Full Certification](#)
15. [Scenario K — Production Readiness Sign-Off](#)
16. [Scenario L — Generate Jira Bugs from Results](#)
17. [Understanding the Outputs](#)
18. [Reading the Coverage Report](#)
19. [Reading Defect Reports — Severity Guide](#)
20. [UI/UX Testing — What the Framework Catches](#)

- 21. [Troubleshooting Common Issues](#)
- 22. [Quick Reference — All Parameters](#)

1. What This Framework Does

The Enterprise Certification Framework is a set of seven Claude Code skills that automate the testing of web applications using a real browser (Playwright/Chromium). It does **not** just take screenshots and compare layouts — it performs real CRUD operations, field validation tests, permission checks, and UI/UX quality checks with evidence captured at every step.

WHAT “CERTIFIED” MEANS

A certification proves all of the following with screenshot evidence:

Dimension	What Was Tested
Page Coverage	Every page in the app was visited and an action was performed
Form Coverage	Every form was submitted with valid data and the success outcome confirmed
Field Coverage	Every field was tested with valid, blank, invalid, boundary, and special-character values
UI/UX Coverage	Submit buttons enable correctly, errors highlight the right field, inline messages appear
Workflow Coverage	Every multi-step workflow was completed on the happy path and at least one error path
Permission Coverage	Every role was tested: accessible pages confirmed, restricted pages confirmed blocked
API Coverage	Every discovered API endpoint was called and its response status verified

WHAT IT IS NOT

- It is NOT a performance or load testing tool
- It is NOT a penetration testing or security exploitation tool
- It is NOT a visual regression comparison tool (it tests behavior, not pixels)
- It does NOT require you to write any test scripts in advance

2. When to Use Each Skill

Skill	Use When
<code>/discover-app</code>	You need an inventory of what's in an app before testing — pages, forms, fields, APIs
<code>/application-certification</code>	You want to fully certify one application URL for one or more roles
<code>/migration-certification</code>	You have an old URL and a new URL and need to confirm the new one works at least as well
<code>/test-data-generator</code>	You have an inventory and need test data generated without running the full certification
<code>/production-readiness-review</code>	You have an existing certification session and need a go/no-go verdict
<code>/generate-jira-bugs</code>	You have certification results and want Jira tickets created for every defect found
<code>/generate-pdf-report</code>	You have certification results and want a formatted report document

Decision tree:

```

Do you have an old system and a new system to compare?
  YES → /migration-certification
  NO  → Do you just want to know what's in the app first?
        YES → /discover-app
        NO  → /application-certification
  
```

3. Prerequisites & Setup

3.1 BROWSE BINARY

All skills require the **paysec** toolkit's `browse` binary (Playwright-backed headless browser). Reports also use paysec's `/md-to-pdf`, and `--final` runs use `/qa-report`, `/plan-tech-review` and `/plan-business-review`.

Check it is installed:

```
! ls ~/.claude/skills/paysec/browser/dist/browse.exe
```

If missing, install paysec first:

```
! git clone <your-paysec-repo-url> ~/.claude/skills/paysec && cd  
~/.claude/skills/paysec && ./setup
```

The binary must exist at: `~/.claude/skills/paysec/browser/dist/browse.exe`
(Windows) `~/.claude/skills/paysec/browser/dist/browse` (Mac/Linux)

Or point `PAYSEC_BROWSE_BIN` at a browse binary elsewhere.

3.2 WORKING DIRECTORY

All session artifacts are written to:

```
{your working directory}/certification-runs/{session-id}/
```

Run your prompts from `D:\claude\CompareSkill\` or any directory where you want the `certification-runs/` folder to appear.

3.3 WHAT YOU NEED BEFORE STARTING

- The URL of the application to test
- Login credentials for each role you want to test

- Whether the app requires special login (SSO, cookie paste, custom script)
- Whether you want destructive operations (Delete) to actually run (or use `dry_run=true` to skip them)

4. How to Write Prompts — The Golden Rules

RULE 1: ALWAYS INCLUDE URL, USERNAME, PASSWORD, ROLE

The minimum viable prompt for any certification:

```
/application-certification url=https://your-app.com username=admin
password=secret role=admin
```

RULE 2: USE `ROLES=` FOR MULTIPLE USERS IN ONE RUN

If you have multiple users to test in the same run, list them with `roles=` and provide their credentials separately in the prompt body:

```
/application-certification url=https://app.com username=admin password=Admin@123
role=admin
roles=merchant,support
merchant_username=merchant1 merchant_password=Merch@123
support_username=support1 support_password=Supp@123
```

RULE 3: USE `--QUICK` DURING DEVELOPMENT, `--FINAL` FOR SIGN-OFF

`--quick` skips the paysec review layer, Jira, and PDF generation — runs 3–5× faster. `--final` enables all outputs — use only for the official go-live decision.

RULE 4: USE `DRY_RUN=TRUE` ON SHARED/PRODUCTION-ADJACENT ENVIRONMENTS

`dry_run=true` skips all Delete, Approve, Reject operations — safe for environments where other people's data must not be touched.

RULE 5: FOR MIGRATION CHECKS, ALWAYS PROVIDE OLD_USERNAME / OLD_PASSWORD IF DIFFERENT

Old and new systems often use different credential schemas. Always be explicit:

```
/migration-certification old_url=https://old.app.com new_url=https://new.app.com  
old_username=admin old_password=OldPass@1 username=admin password=NewPass@1  
role=admin
```

5. Scenario A — Discover a Single Application

WHEN TO USE

- You are unfamiliar with an application and want to see what it contains before testing
- You want a count of all pages, forms, fields, and APIs
- You need to run discovery separately before running certification phases

WHAT HAPPENS

The framework logs in, crawls every page (BFS + SPA-aware routing), clicks every nav item, opens every dialog, and produces `inventory.json` — a machine-readable map of the whole app.

EXAMPLE PROMPTS

Simplest discovery:

```
/discover-app url=https://app.example.com username=admin password=Admin@123
```

Discovery with multiple roles (each role may see different pages):

```
/discover-app url=https://app.example.com username=admin password=Admin@123  
roles=merchant,support  
merchant_username=merchant1 merchant_password=Merch@123  
support_username=support1 support_password=Supp@123
```

Discovery using a pre-authenticated cookie (for SSO apps where you log in manually first):

```
/discover-app url=https://sso-app.example.com login_mode=cookie
session_cookie="sessionId=eyJhb...abc; Path=/; Domain=sso-app.example.com"
```

Resume a previously interrupted discovery:

```
/discover-app url=https://app.example.com username=admin password=Admin@123
resume=true
session_path=certification-runs/20260611_142200_app_example_com
```

WHAT YOU GET

```
certification-runs/20260611_{time}_{slug}/
├─ discovery/
│   ├── inventory.json          ← full structured inventory
│   ├── coverage-map.json      ← initialized coverage tracker
│   ├── session-summary.md     ← human-readable summary
│   └─ screenshots/           ← page screenshots taken during crawl
```

6. Scenario B — Certify One URL, One User

WHEN TO USE

- You want to fully test one application as one role
- Typical for a single-tenant app or a quick check of the admin role only

WHAT HAPPENS

Runs all 12 phases: discovery → test data → form testing → UI/UX testing → CRUD testing → workflow testing → permission testing → coverage calculation → defect aggregation → verdict.

EXAMPLE PROMPTS

Standard run (development mode — fast):

```
/application-certification url=https://app.example.com username=admin  
password=Admin@123 role=admin --quick
```

Full run with all outputs (sign-off mode):

```
/application-certification url=https://app.example.com username=admin  
password=Admin@123 role=admin --final
```

Safe run on shared staging (skip destructive operations):

```
/application-certification url=https://staging.example.com username=admin  
password=Admin@123  
role=admin dry_run=true --quick
```

With slower delay between tests (for rate-limited APIs):

```
/application-certification url=https://app.example.com username=admin  
password=Admin@123  
role=admin test_delay_ms=1500 --quick
```

Resume a run that was interrupted:

```
/application-certification url=https://app.example.com username=admin  
password=Admin@123  
role=admin session_path=certification-runs/20260611_142200_app_example_com --  
quick
```

EXPECTED OUTPUT

APPLICATION CERTIFICATION RESULT			
URL:	https://app.example.com		
Role:	admin		
Score:	94/100		
Verdict:	PASS		
Page Coverage:	100.0%	✓	
Form Coverage:	100.0%	✓	
Field Coverage:	100.0%	✓	
UI/UX Coverage:	88.9%	✗	(1 form: stuck btn)
Workflow Coverage:	100.0%	✓	
Permission Coverage:	100.0%	✓	
API Coverage:	83.3%	✓	
Critical: 0 High: 1 Medium: 2 Low: 1			

7. Scenario C — Certify One URL, Multiple Users/Roles

WHEN TO USE

- Your application has role-based access (admin, merchant, support, viewer, etc.)
- You need to verify that each role sees only what it should
- You need to test that restricted pages/actions are actually blocked

WHAT HAPPENS

Discovery runs once per role (merging inventories). Then certification runs for the primary role with full field and CRUD testing. Permission testing runs for ALL roles, building a permission matrix that cross-checks each role against every page and action.

EXAMPLE PROMPTS

Two roles — admin (primary) + merchant:

```
/application-certification url=https://app.example.com
username=admin password=Admin@123 role=admin
roles=merchant
merchant_username=merchant1 merchant_password=Merch@123
--quick
```

Three roles — admin + merchant + support:

```
/application-certification url=https://app.example.com
username=admin password=Admin@123 role=admin
roles=merchant,support
merchant_username=merchant1 merchant_password=Merch@123
support_username=support1 support_password=Support@123
--quick
```

Four roles with dry_run (no deletions across any role):

```
/application-certification url=https://payments.example.com
username=superadmin password=SuperAdmin@1 role=super_admin
roles=whitelabel,merchant,viewer
whitelabel_username=wl_user1 whitelabel_password=WL@pass1
merchant_username=merchant_user merchant_password=Merch@pass1
viewer_username=view_only viewer_password=View@pass1
dry_run=true --quick
```

WHAT THE PERMISSION MATRIX SHOWS

For each role, you get a complete breakdown:

Role: merchant

Pages accessible (nav): 4 of 12

Pages accessible (direct URL): 4 of 12

Pages blocked (redirect to login): 8 of 12

Row actions visible: Edit, View (2 of 6)

Row actions blocked: Delete, Approve, Reject, Settings (4 of 6)

API endpoints allowed: 8 of 22

API endpoints blocked (403): 14 of 22

Defects raised automatically:

- If `merchant` can reach `/admin/system-settings` → Critical (authorization bypass)
 - If `admin` cannot reach its own pages → High (legitimate access denied)
-

8. Scenario D — Migration Check (Old URL vs New URL, Same Role)

WHEN TO USE

- You have rewritten or migrated an application
- You need to confirm the new version covers everything the old one did
- You want a gap report showing what is missing, changed, or new

WHAT HAPPENS

Runs full certification on BOTH the old and new URLs (every role you provide, on both), checks that both sides were completely tested, then performs a 5-layer behavioral comparison: pass/fail parity, validation message diff, API response structure diff, post-action data state diff, and per-role permission parity. Produces a migration score and gap report.

Full migration sign-off (`--full`) — test both pages completely, with every role:

```

/migration-certification
old_url=https://legacy.example.com/<legacy-page-path>
new_url=https://new.example.com/<new-page-path>
old_scope=/<legacy-page-path> new_scope=/<new-page-path>
username=admin password=Admin@123 role=admin
roles_file=D:/secure/roles.json
shared_db=true
--full

```

`--full` = `--final` + every role on both systems + no dry run + a completeness check that stops the run (status INCOMPLETE, no score) if any form, module, workflow, UI/UX check, or role was not tested on either side. `scope` keeps the run to one page/module; use `old_scope` / `new_scope` when the URLs differ. See [shared/browse-integration.md](#) → [Role Credentials File](#) for the `roles.json` format.

EXAMPLE PROMPTS

Same credentials on both systems:

```

/migration-certification
old_url=https://legacy.example.com
new_url=https://new.example.com
username=admin password=Admin@123
role=admin
--quick

```

Different credentials for old vs new system (most common):

```

/migration-certification
old_url=https://legacy.example.com
new_url=https://new.example.com
old_username=<your-username> old_password=<your-password>
username=<your-username> password=<your-password>
role=super_admin
--quick

```

Migration check with `dry_run` (recommended for production-adjacent old systems):

```

/migration-certification
old_url=https://old-prod.example.com
new_url=https://new-staging.example.com
old_username=admin old_password=OldProd@1
username=admin password=NewStage@1
role=admin
dry_run=true --quick

```

Migration check scoped to a specific section of the app (by providing the old and new sub-URLs):

```

/migration-certification
old_url=https://old.example.com/admin/getAllPaymentBank
new_url=https://new.example.com/banks/all-banks
old_username=<your-username> old_password=<your-password>
username=<your-username> password=<your-password>
role=super_admin
--quick

```

WHAT THE MIGRATION REPORT SHOWS

MIGRATION SCORE: 83/100 – HOLD

Missing features (in old, not in new):

- CRITICAL: Bank Response Mappings (/admin/getAllBankResponseMapping)

Functional regressions (old passed, new fails):

- CRITICAL: Add MID – HTTP 400 DB error on new system

Behavioral drifts (both pass but behavior differs):

- ⚠ Active/Inactive bank count: old=361/41, new=362/40
- ⚠ Search scope: old supports Bank ID search, new is name-only

New features in new system (informational):

- ℹ Inline modal forms replace page navigation
- ℹ Status toggle replaces block/unblock buttons

Migration recommendation thresholds:

- **PROCEED** — score ≥ 100%, no missing features, no regressions

- `PROCEED_WITH_CONDITIONS` — score $\geq 90\%$, no critical gaps
 - `HOLD` — score $< 90\%$ OR any critical gap exists
-

9. Scenario E — Migration Check with Multiple Roles

WHEN TO USE

- You are migrating an application that has role-based access
- You need to confirm that the new system enforces the same role permissions as the old
- You want to verify that a whitelabel/merchant user sees the same restricted view on both systems

EXAMPLE PROMPTS

Migration check: Super Admin + Whitelabel roles:

```
/migration-certification
old_url=https://legacy.example.com
new_url=https://new.example.com
old_username=<your-username> old_password=<your-password>
username=<your-username> password=<your-password>
role=super_admin
roles=whitelabel
old_whitelabel_username=<whitelabel-username> old_whitelabel_password=
<whitelabel-password>
whitelabel_username=<whitelabel-username> whitelabel_password=<whitelabel-
password>
--quick
```

Three-role migration with dry_run:

```
/migration-certification
old_url=https://legacy-bank.example.com
new_url=https://new-bank.example.com
old_username=sa_user old_password=SA@Pass1
username=sa_user password=SA@Pass1
role=super_admin
roles=whitelabel,merchant
old_whitelabel_username=w11 old_whitelabel_password=WL@Pass1
whitelabel_username=w11 whitelabel_password=WL@Pass1
old_merchant_username=merch1 old_merchant_password=Merch@Pass1
merchant_username=merch1 merchant_password=Merch@Pass1
dry_run=true --quick
```

ROLE PARITY CHECK OUTPUT

For each role, the framework generates a role parity table:

Role: whitelabel	
Old URL → Banks visible: 1 (hdfc dkp) 	New URL → Banks visible: 1 (hdfc dkp)
Old URL → Add Bank:  hidden 	New URL → Add Bank:  hidden
Old URL → Delete:  hidden 	New URL → Delete:  hidden
Old URL → MID row actions: 2	New URL → MID row actions: 6
 New has more	

10. Scenario F — Test a Complete Application (Multiple Base URLs)

WHEN TO USE

- Your application has separate URL bases for different modules (e.g., login at `auth.company.com`, admin at `admin.company.com`, API at `api.company.com`)
- You need to certify each module separately and then consolidate the report

APPROACH

Run separate `/application-certification` calls per base URL, then use `session_path` references to consolidate into a single `/production-readiness-review`.

EXAMPLE PROMPTS

Module 1 — Authentication & Login module:

```
/application-certification url=https://auth.company.com
username=admin@company.com password=Admin@123
role=admin --quick
```

Module 2 — Admin dashboard (different base URL):

```
/application-certification url=https://admin.company.com
username=admin@company.com password=Admin@123
role=admin --quick
```

Module 3 — Merchant portal:

```
/application-certification url=https://merchant.company.com
username=merchant1@company.com password=Merch@123
role=merchant --quick
```

Consolidated production readiness review after all three:

```
/production-readiness-review
session_path=certification-runs/20260611_auth_company_com,certification-
runs/20260611_admin_company_com,certification-runs/20260611_merchant_company_com
```

11. Scenario G — Test Specific Pages Only

WHEN TO USE

- You don't want to certify the whole application — only specific pages or modules

- You have already run discovery and want to re-run tests on a subset
- You made changes to one module and want to re-certify only that module

APPROACH

Run `/discover-app` first to get the inventory, then run `/application-certification` with `session_path` pointing to the existing session — it will skip discovery and let you focus test data generation and testing on the specific forms/tables you specify.

EXAMPLE PROMPTS

First — discover the full app once:

```
/discover-app url=https://app.example.com username=admin password=Admin@123
```

Note the session path from the output: `certification-runs/20260611_142200_app_example_com`

Then — certify only the banks module (by targeting its page URL directly):

```
/application-certification url=https://app.example.com/banks/all-banks
username=admin password=Admin@123 role=admin
session_path=certification-runs/20260611_142200_app_example_com
--quick
```

Or certify a specific form by URL:

```
/application-certification url=https://app.example.com/merchants/create
username=admin password=Admin@123 role=admin --quick
```

Or re-run only the UI/UX checks on a specific page after a fix:

```
Please run only the UI/UX testing phase (Phase 3b from application-
certification/references/ui-ux-testing.md)
on the form at https://app.example.com/banks/add-bank using username=admin
password=Admin@123.
Save results to certification-runs/20260611_banks_uiux_recheck/
```

12. Scenario H — Read-Only / Safe Check (No Data Mutations)

WHEN TO USE

- You are testing against a production-adjacent or shared staging environment
- Other teams are actively using the environment and you cannot create/delete records
- You want to verify that pages load and forms are present, but not submit anything

EXAMPLE PROMPTS

Full certification — skip all destructive operations:

```
/application-certification url=https://staging.example.com
username=admin password=Admin@123 role=admin
dry_run=true --quick
```

Migration check — read-only on old system, mutations allowed on new:

```
/migration-certification
old_url=https://production.example.com
new_url=https://staging.example.com
old_username=readonly_admin old_password=ReadOnly@1
username=admin password=Admin@123
role=admin
dry_run=true --quick
```

WHAT DRY_RUN SKIPS

Action	dry_run=false	dry_run=true
Create record	✓ Executes	✓ Executes
Edit record	✓ Executes	✓ Executes
View record	✓ Executes	✓ Executes
Delete record	✓ Executes	▶▶ Skipped (marked <code>skipped_dry_run</code>)
Approve/Reject	✓ Executes	▶▶ Skipped
Deactivate/Block	✓ Executes	▶▶ Skipped

13. Scenario I — Resume an Interrupted Run

WHEN TO USE

- A long certification was interrupted (browser crash, network drop, session timeout)
- You want to continue from where it stopped, not start from scratch

HOW CHECKPOINTS WORK

The framework writes `checkpoint.json` after every major completed phase. On the next run with the same `session_path`, it reads the checkpoint and skips already-completed phases.

EXAMPLE PROMPTS

Resume from the last checkpoint:

```
/application-certification url=https://app.example.com
username=admin password=Admin@123 role=admin
session_path=certification-runs/20260611_142200_app_example_com
--quick
```

The framework will print:

```
Checkpoint found. Resuming from phase: crud_testing
Skipping: discovery, test_data_generation, form_testing, ui_ux_testing
Continuing: crud_testing → workflow_testing → permission_testing → ...
```

Resume a migration certification:

```
/migration-certification
old_url=https://legacy.example.com
new_url=https://new.example.com
old_username=admin old_password=Old@123
username=admin password=New@123
role=admin
session_path=certification-runs/20260611_migration_new_example_com
--quick
```

14. Scenario J — Quick Check vs Full Certification

FLAGS THAT CONTROL THE RUN DEPTH

Flag	What It Enables	When to Use
(no flag)	Core certification only	Daily development testing
<code>--quick</code>	Same as no flag — explicitly skip review layer	Same as above, explicit intent
<code>--final</code>	+ paysec review (<code>/qa-report</code> , <code>/plan-tech-review</code> , <code>/plan-business-review</code>) + Jira + PDF	Official go/no-go decisions

DEVELOPMENT CYCLE PROMPTS

Daily check — fast, just core results:

```
/application-certification url=https://dev.example.com username=dev
password=Dev@123 role=admin --quick
```

Pre-release check — full certification, no review:

```
/application-certification url=https://staging.example.com username=admin  
password=Staging@1 role=admin
```

Go-live sign-off — full certification + expert review + Jira + PDF:

```
/application-certification url=https://staging.example.com username=admin  
password=Staging@1 role=admin  
roles=merchant,support  
merchant_username=merch1 merchant_password=Merch@1  
support_username=sup1 support_password=Sup@1  
--final
```

15. Scenario K — Production Readiness Sign-Off

WHEN TO USE

- Certification has already been run
- You need a formal go/no-go verdict with all gates evaluated
- You want the production readiness review run separately from the certification

EXAMPLE PROMPTS

Run production readiness review on an existing certification session:

```
/production-readiness-review session_path=certification-  
runs/20260611_142200_app_example_com
```

Run fresh certification + immediately review:

```
/application-certification url=https://app.example.com username=admin  
password=Admin@123  
role=admin --final
```

(The `--final` flag invokes production readiness review automatically at the end.)

HARD GATES THAT MUST ALL PASS

- ✓ GATE_PAGE_COV – All pages visited + action taken
- ✓ GATE_FORM_COV – All forms submitted with valid data + success confirmed
- ✓ GATE_FIELD_COV – All fields tested with all applicable test types
- ✓ GATE_UIUX_COV – UI/UX checks passed (or acknowledged with justification)
- ✓ GATE_WORKFLOW_COV – All workflows: success + failure paths executed
- ✓ GATE_PERMISSION_COV – All roles × resources tested
- ✓ GATE_API_COV – $\geq 80\%$ of discovered endpoints called + status verified
- ✓ GATE_CRITICAL_DEFECTS – 0 critical defects
- ✓ GATE_EVIDENCE – Every test result has ≥ 1 screenshot

If ANY gate fails, verdict = FAIL. There is no partial pass.

16. Scenario L — Generate Jira Bugs from Results

WHEN TO USE

- Certification found defects
- You want to create Jira tickets automatically for each defect
- You want a CSV export for manual import

EXAMPLE PROMPTS

Export-only mode (no Jira credentials — generates CSV + JSON):

```
/generate-jira-bugs session_path=certification-
runs/20260611_142200_app_example_com
```

Live Jira push (requires environment variables):

First set your Jira credentials:

```
! $env:JIRA_URL = "https://yourorg.atlassian.net"
! $env:JIRA_API_TOKEN = "your-base64-encoded-user:token"
! $env:JIRA_PROJECT_KEY = "CERT"
```

Then run:

```
/generate-jira-bugs session_path=certification-
runs/20260611_142200_app_example_com
```

Or pass credentials inline:

```
/generate-jira-bugs session_path=certification-
runs/20260611_142200_app_example_com
jira_url=https://yourorg.atlassian.net jira_api_token=base64token
jira_project_key=CERT jira_labels=certification,auto-generated
```

OUTPUT FILES

```
defects/
├─ defects.json          ← All defects in framework schema
├─ jira-bugs.json        ← Jira-shaped issue objects (ADF format)
├─ jira-bugs.csv         ← CSV for manual Jira import
└─ jira-issue-keys.json ← (only if live push) Maps defect IDs to Jira keys
```

17. Understanding the Outputs

Every certification run creates this directory structure:

```

certification-runs/{session-id}/
|
├─ checkpoint.json          ← Resume state (which phase completed last)
|
├─ discovery/
|   └─ inventory.json      ← Full app map: pages, forms, fields, tables,
                             APIs, workflows
|   └─ coverage-map.json   ← Coverage tracker (updated as tests run)
|       └─ screenshots/    ← Screenshots taken during crawl
|
├─ test-data/
|   └─ {form-name}.json    ← Generated test cases per form (10 types per
                             field)
|
├─ test-results/
|   └─ form-tests/         ← Field-by-field test results per form
|   └─ crud-tests/        ← CRUD operation results per module
|   └─ ui-ux-tests/       ← UI/UX check results per form (9 checks)
|   └─ workflow-tests/    ← Workflow execution results
|       └─ permission-tests/ ← Permission matrix per role
|
├─ playwright-regression/
|   └─ {form-name}.spec.ts ← Replayable Playwright scripts for CI/CD
|
├─ evidence/
|   └─ screenshots/       ← All test screenshots (before/after/error)
|   └─ api-logs/          ← Captured API requests and responses
|       └─ console-logs/  ← Browser console output during tests
|
├─ coverage/
|   └─ page-coverage.json
|   └─ form-coverage.json
|   └─ field-coverage.json
|   └─ ui-ux-coverage.json ← Added in v2.0
|   └─ workflow-coverage.json
|   └─ permission-coverage.json
|       └─ api-coverage.json
|
├─ defects/
|   └─ defects.json        ← All defects with severity and evidence
|   └─ jira-bugs.json
|       └─ jira-bugs.csv
|
├─ reports/
|   └─ executive-summary.pdf ← (or .html if paysec /md-to-pdf unavailable)

```

```

|   | functional-testing.pdf
|   | defect-report.pdf
|   | traceability-matrix.pdf ← Only for migration runs
|   | coverage-report.pdf
|   | production-readiness.pdf
|   | test-case-register.pdf ← EVERY test case with input, expected, actual,
result, evidence
|                                   (migration runs: legacy vs new comparison +
both full lists)
|
| certification-result.json ← Final verdict, score, gate evaluation

```

18. Reading the Coverage Report

The coverage report (`coverage-report.pdf`) contains a dashboard table like this:

Dimension	Total Discovered	Covered	Acknowledged	Coverage %	Gate
Pages	12	12	0	100.0%	✓
Forms	8	7	0	87.5%	✗
Fields	47	47	0	100.0%	✓
UI/UX Checks	72	63	0	87.5%	✗
Workflows	3	3	0	100.0%	✓
Permissions	24	24	0	100.0%	✓
API Endpoints	22	18	0	81.8%	✓ (≥80%)

Items in the “Not Covered” section are listed explicitly with their ID, type, and the reason they were not covered (could not reach, blocked by auth, no test data, etc.).

Acknowledged Unreachable items are covered items that were explicitly waived. Each must have a written justification. Maximum 10% of any dimension can be waived without requiring human sign-off.

19. Reading Defect Reports — Severity Guide

CRITICAL — BLOCKS GO-LIVE UNCONDITIONALLY

These are show-stoppers. Even one Critical defect = FAIL verdict.

Example Defect	What It Means
Email field accepts <code>notanemail</code>	Form submits and saves invalid data
Merchant can reach <code>/admin/delete-all</code>	Authorization bypass — security issue
Create form always returns 500	Core feature completely broken
Delete happens without any confirmation	Data loss risk

HIGH — MUST BE FIXED; DOES NOT AUTOMATICALLY BLOCK IF TRACKED

Example Defect	What It Means
Submit button stays disabled after valid form fill	Users cannot complete the form
Field not highlighted on validation error	Users cannot identify which field to fix
Add MID returns HTTP 400 DB error	Feature non-functional on the backend
Edit form shows success but data reverts on reload	Data not persisted

MEDIUM — SHOULD BE FIXED; DOCUMENT AS KNOWN ISSUE IF DEFERRED

Example Defect	What It Means
Only a toast error shown, no inline field error	Users must hunt for what went wrong
Required field has no * marker	Users don't know what is required
Error message says "Invalid" with no detail	Not actionable for the user
Search returns blank screen instead of "No results"	Confusing empty state

LOW — NICE TO FIX; NO PRODUCTION RISK

Example Defect	What It Means
Placeholder is the only label (disappears on type)	Minor usability issue
Read-only field looks identical to editable field	Visual clarity issue
"Businiess Name" typo in label	Cosmetic

20. UI/UX Testing — What the Framework Catches

The UI/UX phase (Phase 3b) runs 9 checks per form automatically. Here is what each check looks for and why it matters:

CHECK 1 — REQUIRED FIELD INDICATORS

What: Every required field has a visible * or equivalent marker.

Why: Without it, users submit and get mysterious errors, not knowing which fields are required.

Example defect: "Email field is required but has no visual marker — users discover this only after failed submission."

CHECK 2 & 3 — SUBMIT BUTTON STATE

What: Button is appropriately disabled on blank form; enables when all required fields are validly filled.

Why: If the button never enables after valid fill, the user is stuck with no feedback.

Example defect: “Add Bank form: Confirm button stays disabled after all required fields are filled with valid data.”

CHECK 4 — STUCK FORM DIAGNOSIS

What: When Check 3 fails, automatically identifies WHICH field is blocking the button.

Why: Without this, a defect report just says “form doesn’t work” — useless. The framework names the exact field.

Example output: “Blocking fields: [{field: ‘postalCode’, reason: ‘aria-invalid=true’}, {field: ‘currency’, reason: ‘required_still_empty’}]”

CHECK 5 — FIELD HIGHLIGHT ON ERROR

What: After a bad submit (blank required fields or invalid values), the problem fields must turn red or show `aria-invalid`.

Why: Without visual highlighting, the user sees a generic error message and cannot tell which of 10 fields is wrong.

Example defect: “Create Merchant form: 0 fields highlighted after submit with empty required fields. User has no visual indicator of which fields to fix.”

CHECK 6 — ERROR MESSAGE QUALITY

What: Error messages must be descriptive — not just “Error” or “Invalid”.

Why: “Invalid format” on an 8-field form tells the user nothing. “Email address must be in the format [user@domain.com](#)” is actionable.

Example defect: “Auth Key field shows error message ‘Invalid’ (3 characters) — too vague to guide the user.”

CHECK 7 — LABEL PRESENCE

What: Every field has a persistent label (not placeholder-only).

Why: Placeholder text disappears when the user starts typing. In a long form, the user loses context of what they are filling.

Example defect: “Postal Code field has no label — only a placeholder. Once the user starts typing, the field purpose is invisible.”

CHECK 8 — EMPTY STATE MESSAGE

What: Search/filter results that return zero items show a human-readable message.

Why: A blank table with no message looks like a loading error or a bug.

Example defect: “Bank list shows blank page when search returns no results — no ‘No banks found’ message.”

CHECK 9 — DISABLED FIELD CLARITY

What: Read-only fields look different from editable fields.

Why: In a view-only edit dialog, if disabled fields look identical to editable ones, users try to edit them and are confused when nothing happens.

Example defect: “WL Edit Bank dialog: disabled fields have same white background and cursor as editable fields — visually indistinguishable.”

21. Troubleshooting Common Issues

”BROWSE BINARY NOT FOUND”

```
BLOCKED: paysec browse binary not found.
```

Fix: Run the paysec setup:

```
! cd ~/.claude/skills/paysec && ./setup
```

”SESSION EXPIRED MID-RUN”

The browser session token expired (common after ~5 minutes of inactivity on React apps).

Fix: The framework automatically re-logs in every 20 tests (see `field-testing-protocol.md §Step 12`). If this doesn't work for your app, increase `test_delay_ms` to reduce test frequency, or use `login_mode=cookie` with a long-lived session cookie.

"SUBMIT BUTTON STAYS DISABLED (STUCK FORM)"

The framework will automatically diagnose this as a High defect and identify the blocking fields. You do not need to investigate manually — read the defect report for the `stuck_fields` array.

If you encounter this during manual testing (not framework), check:

1. Is there a required dropdown that wasn't selected?
2. Is there a field with a hidden validation error (aria-invalid but no visible message)?
3. Is there a CAPTCHA or token field that the automation cannot fill?

"INVENTORY.JSON NOT PRODUCED"

Discovery failed before completing. Check:

1. Login succeeded — verify the session actually reached the logged-in state
2. The URL is reachable — try `browse goto {url}` manually
3. The app is not returning a CAPTCHA after login

"FORM TEST FAILS EVEN WITH VALID DATA"

Common cause: React-style inputs (controlled components) don't update when `browse fill` sets the DOM value directly — the React state doesn't register the change.

Fix: Use the React-compatible setter pattern. The framework handles this automatically, but if writing a manual test:

```
var setter = Object.getOwnPropertyDescriptor(window.HTMLInputElement.prototype,
'value').set;
setter.call(inputElement, 'your value');
inputElement.dispatchEvent(new Event('input', {bubbles: true}));
```

”RATE LIMITED — 429 RESPONSES”

The app’s API is rate-limiting the test traffic.

Fix: Increase the delay between tests:

```
/application-certification url=... username=... password=... role=admin  
test_delay_ms=2000
```

The framework automatically backs off exponentially on 429s (2s, 4s, 8s, 16s, up to 5 retries).

”CERTIFICATION SHOWS 0% UI/UX COVERAGE”

Phase 3b (UI/UX testing) was not run. This happens if:

1. You resumed from a checkpoint that predates the UI/UX phase being added
2. The discovery produced no forms

Fix: Delete the checkpoint and re-run:

```
! Remove-Item certification-runs/{session-id}/checkpoint.json  
/application-certification url=... username=... password=... role=admin  
session_path=certification-runs/{session-id} --quick
```

22. Quick Reference — All Parameters

/DISCOVER-APP

Parameter	Type	Required	Default	Description
<code>url</code>	string	Yes	—	Root URL to crawl
<code>username</code>	string	Yes	—	Login username
<code>password</code>	string	Yes	—	Login password
<code>login_mode</code>	enum	No	<code>form</code>	<code>form</code> / <code>cookie</code> / <code>script</code>
<code>session_cookie</code>	string	No	—	Pre-auth cookie (cookie mode)
<code>login_script_path</code>	string	No	—	Custom login instructions (script mode)
<code>roles</code>	string	No	—	Comma-separated additional roles
<code>resume</code>	bool	No	<code>false</code>	Resume from existing checkpoint

/APPLICATION-CERTIFICATION

Parameter	Type	Required	Default	Description
<code>url</code>	string	Yes	—	Application root URL
<code>username</code>	string	Yes	—	Primary role username
<code>password</code>	string	Yes	—	Primary role password
<code>role</code>	string	Yes	—	Primary role name
<code>login_mode</code>	enum	No	<code>form</code>	<code>form</code> / <code>cookie</code> / <code>script</code>
<code>dry_run</code>	bool	No	<code>false</code>	Skip destructive operations
<code>test_delay_ms</code>	int	No	<code>500</code>	Delay between test submissions (ms)
<code>roles</code>	string	No	—	Additional roles for permission testing
<code>session_path</code>	string	No	—	Resume existing session
<code>--quick</code>	flag	No	—	Skip paysec review + Jira + PDF
<code>--final</code>	flag	No	—	Enable all outputs (sign-off mode)

/MIGRATION-CERTIFICATION

Parameter	Type	Required	Default	Description
<code>old_url</code>	string	Yes	—	Legacy application URL
<code>new_url</code>	string	Yes	—	New application URL
<code>username</code>	string	Yes	—	New system username
<code>password</code>	string	Yes	—	New system password
<code>old_username</code>	string	No	<code>username</code>	Override username for old system
<code>old_password</code>	string	No	<code>password</code>	Override password for old system
<code>role</code>	string	Yes	—	Primary role to certify
<code>login_mode</code>	enum	No	<code>form</code>	Login mode for both systems
<code>dry_run</code>	bool	No	<code>false</code>	Skip destructive operations
<code>roles</code>	string	No	—	Additional roles
<code>--quick</code>	flag	No	—	Skip review + PDF
<code>--final</code>	flag	No	—	Full output (sign-off mode)

/PRODUCTION-READINESS-REVIEW

Parameter	Type	Required	Description
<code>session_path</code>	string	Yes	Path to existing certification session

/GENERATE-JIRA-BUGS

Parameter	Type	Required	Default	Description
<code>session_path</code>	string	Yes	—	Certification session path
<code>jira_url</code>	string	No	<code>\$JIRA_URL</code> env	Jira instance URL
<code>jira_api_token</code>	string	No	<code>\$JIRA_API_TOKEN</code> env	Base64 user:token
<code>jira_project_key</code>	string	No	<code>\$JIRA_PROJECT_KEY</code> env	Jira project key
<code>jira_issue_type</code>	string	No	Bug	Issue type name
<code>jira_labels</code>	string	No	certification	Comma-separated labels

Appendix — Certification Health Score Formula

```
score = (page_pct + form_pct + field_pct + uiux_pct + workflow_pct +
permission_pct + api_pct) / 7
```

Where each `_pct` = $(\text{covered} + \text{acknowledged}) / \text{total} * 100$

```
critical_penalty = critical_defects × 5
high_penalty     = high_defects × 2
```

```
adjusted_score = max(0, raw_score - critical_penalty - high_penalty)
```

The adjusted score is what appears in reports. A score of 100 with zero defects is the only path to a clean PASS.

Appendix — Migration Score Formula

```
migration_score = (equivalent_features / total_old_features) × 100
```

Where:

equivalent_features = features in old system that have a working equivalent in new system

total_old_features = pages + forms + workflows in old system

PROCEED = score ≥ 100% AND no missing features AND no critical regressions

PROCEED_WITH_CONDITIONS = score ≥ 90% AND no critical gaps

HOLD = score < 90% OR any critical gap OR any critical regression

Enterprise Certification Framework | D:\claude\CompareSkill

Skills: /discover-app · /application-certification · /migration-certification

· /test-data-generator · /production-readiness-review · /generate-jira-bugs · /generate-pdf-report